

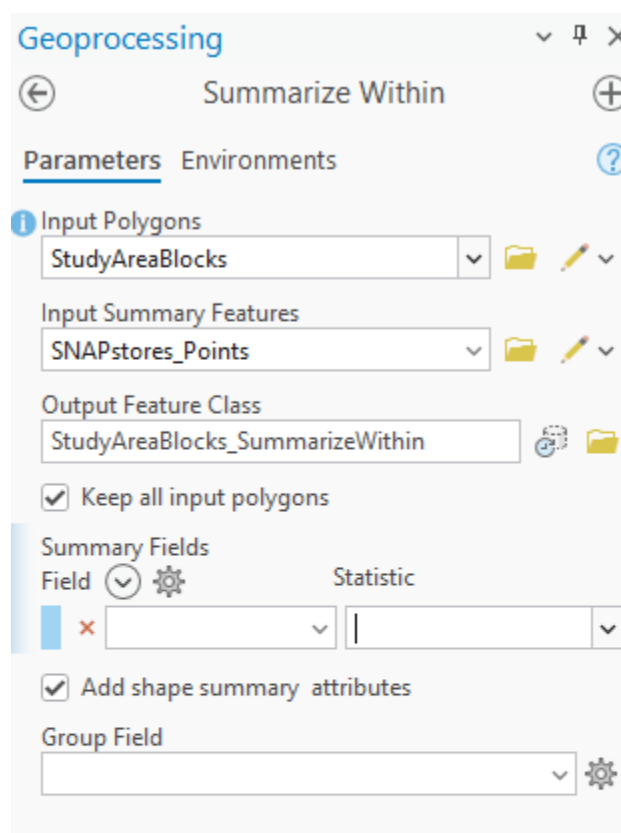
ABQ Transit Access Dashboard Workflow Tutorial

This document will explain how to calculate accessibility isochrones using ArcGIS tools and update the Transit Access Dashboard.

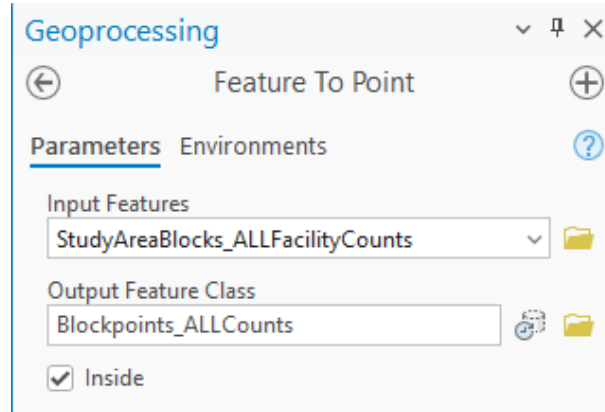
Github [Repository](#)

1. Data requirements
2. ArcGIS workflow
3. Updating Transit Access Dashboard with new data
4. Optional: adding additional accessibility metrics
5. Current data sources and JavaScript libraries

1. The accessibility analysis requires the following datasets:
 - a. Census block shapefile
 - b. GTFS data (text file)
 - c. Road centerline shapefile
 - d. Accessibility metric data
2. ArcGIS Workflow
 - a. Step 1 - Joining accessibility metric data to the census block shapefile
 - i. Add all accessibility datasets to the content pane
 1. Note: depending on data format, you may need to [geocode](#) the addresses
 - ii. Use the summarize within tool to get counts of all accessibility metrics
 1. Iterate the summarize within for each accessibility metric
 - Leave all summary fields and group field blank



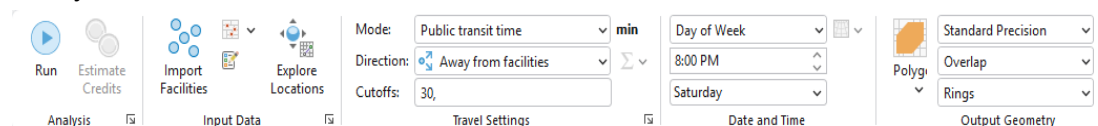
- b. Step 2 - Generate centroids for each block using the feature to point tool (be sure “inside” is checked- this will ensure the points are generated within their respective blocks.



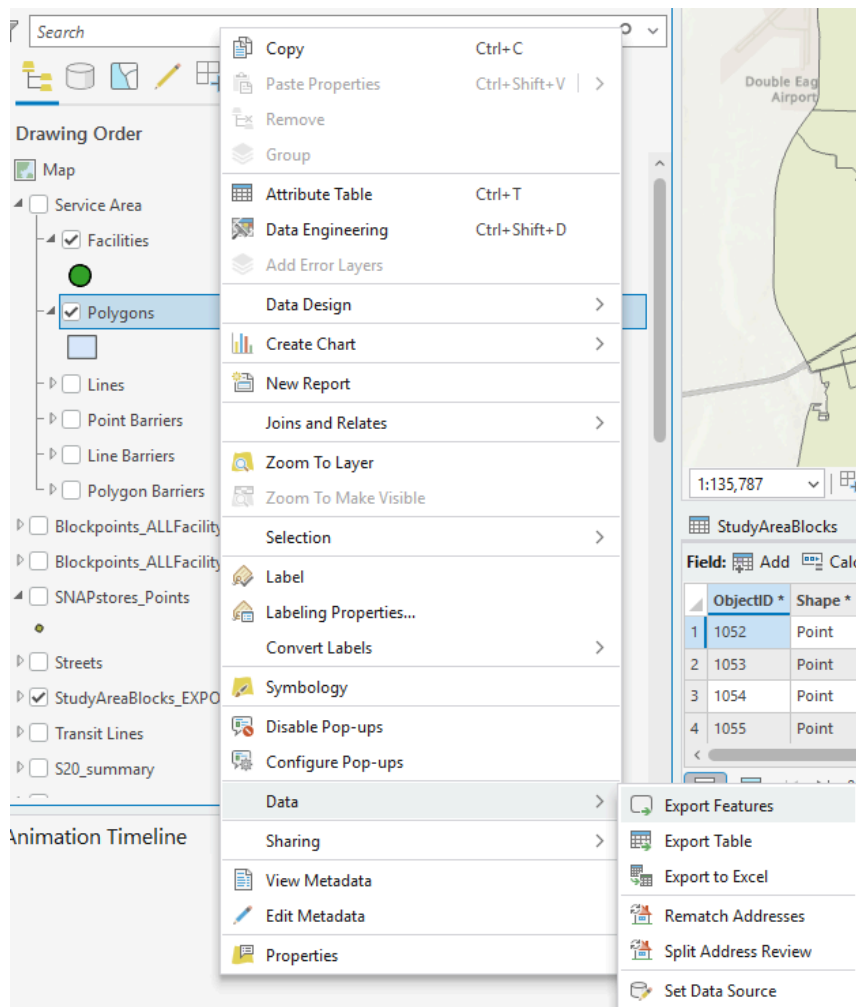
- i. The final product should be a feature layer with points for each block containing a count of each metric

GISJOIN	NAME	Job_Count	School_Count	SNAP_Count	Hospital_Count
G35000100007182002	Block 2002	1446	0	3	1
G35000100024031000	Block 1000	13	0	3	0
G35000100045022015	Block 2015	0	0	3	0
G35000100047223037	Block 3037	117	0	3	0

- c. Step 3 - prepare network dataset
- Follow this [network dataset with public transit data tutorial](#) (pdf of tutorial and network configuration xml are included in tutorial file for this dashboard)
 - When prompted to add the field “RestrictPedestrians” use the field calculator to output “Y” when speed is over 60. This is a fairly good approximation of roads within Albuquerque where there are not sidewalks.
 - When prompted in the tutorial to add facilities, click import facilities and use the previously created blockpoints feature that contains the accessibility count as facilities.
- d. Step 4- generate isochrones
- In the service area layer, enter desired cutoff time, day of week and time of day. And click run



- ii. After the polygons are created, right click on the polygons layer of the service area and export features



- iii. Change the date and time parameters and run again for each of your desired timeframes.

- iv. For each polygon layer, use the summarize within tool to “sum” the count of all accessibility fields in the block points layer

Geoprocessing

Summarize Within

Parameters Environments

Input Polygons
Polygons

Input Summary Features
Blockpoints_ALLFacilityCounts

Output Feature Class
S8_summary

☒ Keep all input polygons

Summary Fields

Field	Statistic
Job_Count	Sum
School_Count	Sum
SNAP_Count	Sum
Hospital_Count	Sum

☐ Add shape summary attributes

Group Field

- v. The dashboard script expects a specific naming convention for the geojson files and field names

```
schools: {  
  label: "School Access",  
  type: "scenario",  
  fields: {  
    weekday_peak: "M8_School_Count",  
    weekday_offpeak: "M14_School_Count",  
    weekday_evening: "M20_School_Count",  
    weekend_peak: "S8_School_Count",  
    weekend_offpeak: "S14_School_Count",  
    weekend_evening: "S20_School_Count"  
  }  
}
```

- vi. Ensure that for each accessibility metric, the naming convention follows this pattern: “M8_School_Count” where M8 refers to Monday at 8 am.

Ensure that all accessibility fields across all timeframe polygons match this structure. EX: "S14_Hospital_Count" for Saturday 2 pm hospital count
"M20_Job_Count", "M8_SNAP_Count"

- vii. For each time period isochrone, use the join field tool to join the accessibility counts from the isochrone polygons to the original block polygon shapefile.
- viii. The final product should be 1 census block feature layer with all accessibility counts for each time period and 6 isochrone feature layers

M8_Job_Count	M8_School_Count	M8_SNAP_Count	M8_Hospital_Count	M14_Job_Count	M14_School_Count	M14_SNAP_Count	M14_Hospital_Count	M20_Job_Count
545	0	1	0	545	0	1	0	545
3742	3	8	3	6365	4	9	2	657
575	0	1	0	6274	4	9	2	575
3740	3	8	3	6274	4	9	2	624

- ix. Use the features to JSON tool to export each of these feature layers (be sure to select "output to GeoJSON" and "Project to WGS84"

Features To JSON

Parameters Environments ?

Input Features

M20_summary_ExportFeatures

Output JSON

C:\Users\cgibb\OneDrive - University of New Me

☒ Formatted JSON

☐ Include Z Values

☐ Include M Values

☒ Output to GeoJSON

☒ Project to WGS84

☐ Use field aliases

3. Updating Transit Access Dashboard with new data
 - a. Converting data to topoJSON - due to the large size of the data involved, it is necessary to convert the data to topoJSON to simplify the geometry of the isochrones and block layers.
 - i. Utilize [this tool](#) to convert all GeoJSON to TopoJSON. (if this tool is unavailable, other open source tools are available)
 - b. Replace all 6 files in the data folder with the new TopoJSON files
 - c. Update file names in the JavaScript file
 - i. Choropleth (census blocks)

```
5  const CONFIG = {
6    data: {
7      //choropleth map file name
8      blocksUrl: "data/blockfinal.topojson",
9      isochroneFolder: "data/"
10   }
```

- ii. The script expects the isochrones files with the following naming convention:
Monday 8 AM= weekday_peak.topojson
Monday 2 PM= weekday_offpeak.topojson
Monday 8 AM= weekday_evening.topojson
Monday 8 AM= weekend_peak.topojson
Monday 8 AM= weekend_offpeak.topojson
Monday 8 AM= weekend_evening.topojson

4. Optional: adding additional metrics- adding additional metrics to the dashboard (or replacing existing ones) is a straightforward process
 - a. In the "metrics" section of JavaScript duplicate a block from another metric:

```
41  jobs: {
42    label: "Jobs Access",
43    type: "scenario",
44    fields: {
45      weekday_peak: "M8_Job_Count",
46      weekday_offpeak: "M14_Job_Count",
47      weekday_evening: "M20_Job_Count",
48      weekend_peak: "S8_Job_Count",
49      weekend_offpeak: "S14_Job_Count",
50      weekend_evening: "S20_Job_Count"
51    },
52    formatter: value => formatNumber(value, 0)
53  },
```

- b. Update the property line, label line and field names to reflect the new fields.
- c. In the metric selection section of the HTML file, duplicate a line from another metric

```

266 <!-- Metric Selection -->
267 <section class="panel-section" id="metric-panel">
268   <!-- <h3 class="section-title">Metric Selection</h3> -->
269   <div class="form-group mb-3">
270     <label for="metricSelect" class="form-label">Choose Accessibility Metric</label>
271     <select id="metricSelect" class="form-select">
272       <option value="svi" selected>Social Vulnerability Index (SVI)</option>
273       <option value="jobs">Jobs Access</option>
274       <option value="healthcare">Healthcare Access</option>
275       <option value="schools">School Access</option>
276       <option value="food">Food Access</option>

```

- d. Update the value field to match the property line from the new metric.

5. Current Data Sources and JavaScript Library Links

a. Current Data Sources:

[Road Network](#)

[GTFS data](#)

[Census Block and Boundary Data](#)

[CDC SVI](#)

[LEHD Survey](#)

[ABQ Public and Charter Schools Data](#)

[USDA SNAP Retailers](#)

[Department of Health and Human Services Medicaid Providers](#)

b. JavaScript Library Links

[Bootstrap](#)

[Leaflet](#)

[Leaflet Control Geocoder](#)

[TopoJSON](#)

[Turf](#)

[D3](#)

[Simple Statistics](#)